## 3.4 Manual Code Review — Titik Fokus

Lakukan code review manual pada file-file berikut (titik rawan historis):

| File | Alasan Review |
|---|---|
| classes/security/authorization/ | Logic otorisasi |
| lib/pkp/classes/db/DAO.inc.php | Akses database, potential SQL injection |
| classes/submission/SubmissionFileManager.inc.php | File upload validation |
| plugins/generic/ | Plugin pihak ketiga, attack surface luas |
| lib/pkp/classes/template/PKPTemplateManager.inc.php | Template rendering, XSS risk |

1. classes/security/authorization/

   **OjsIssueGalleyRequiredPolicy**

   **ID di-cast ke integer:**

   $issueGalleyId = (int)$this->getDataObjectId();

   ✔️ **AMAN → mencegah injection dari input**

   **Validasi object:**

   if (!is_a($issueGalley, 'IssueGalley')) return AUTHORIZATION_DENY;

   ✔️ memastikan object valid

**Validasi relasi:**

$issueGalleyDao->getById($issueGalleyId, $issue->getId());

✔️ memastikan galley milik issue yang benar

**Output:**

**Sudah menerapkan defensive programming. Tidak ada bypass langsung.**

**Authorization policy ini telah melakukan validasi ID dengan casting integer dan memastikan relasi data (issue galley dengan issue). Tidak ditemukan indikasi bypass atau insecure direct object reference (IDOR).**

**OjsIssueRequiredPolicy**

**✔️ Validasi ID**
**if (!$issueId) return AUTHORIZATION_DENY;**

**✔️ Validasi kepemilikan data**
**$issueDao->getByBestId($issueId, $this->journal->getId());**

**✔️ Role-based access control**
**array_intersect($userRoles, [...])**

**POTENSI RISIKO:**

## Input dari user:

**$this->_request->getUserVar($this->_parameterName)**
**➡️ ini berasal dari user input (GET/POST)**

**Tapi:**
**ctype_digit(...)**
**✔️ sudah divalidasi numeric → relatif aman**

**Policy ini menerapkan validasi input dan role-based access control dengan baik. Meskipun data berasal dari user input, penggunaan ctype_digit() memastikan hanya nilai numerik yang diproses, sehingga mengurangi risiko injection.**

**OjsJournalMustPublishPolicy**

**✔️ Context check**

**if (!$this->_context) return AUTHORIZATION_DENY;**

✔️ **Role bypass (intended)**
**ROLE_ID_MANAGER, ROLE_ID_SITE_ADMIN, …**
**bukan bug → desain sistem**

✔️ **Business logic:**
**if ($this->_context->getData('publishingMode') == PUBLISHING_MODE_NONE)**
**jurnal tidak publish → deny**

**Bergantung pada:**
**getAuthorizedContextObject()**
**→ kalau upstream salah, bisa risk**

**Policy ini mengatur akses berdasarkan status publishing journal dan role pengguna. Tidak ditemukan kerentanan langsung, namun keamanan bergantung pada integritas context dan role assignment.**

Manual code review pada modul authorization menunjukkan bahwa sistem telah menerapkan validasi yang cukup baik, termasuk:

- Validasi input dengan casting dan pengecekan numerik
- Validasi object untuk memastikan keberadaan data
- Role-based access control untuk membatasi akses
- Validasi relasi antar entitas (issue dan galley)

Tidak ditemukan kerentanan kritis seperti SQL Injection atau authorization bypass secara langsung.

Namun, terdapat ketergantungan pada input user (getUserVar) dan context authorization yang jika tidak divalidasi secara konsisten di layer lain dapat berpotensi menimbulkan celah keamanan.

2. lib/pkp/classes/db/DAO.inc.php

✔️ **Menggunakan parameter binding**
**return Capsule::cursor(Capsule::raw($sql), $params);**

**Query dipisah dengan:**

- **$sql**
- **$params**

➡️ **Ini artinya:**
**Prepared statement digunakan**

✔️ **Contoh aman di sistem**

**$this->retrieve($sql, $params);**
**➡️ Parameter tidak langsung di-concat ke query**

**Pendekatan ini mengurangi risiko SQL Injection karena input user tidak langsung dimasukkan ke dalam query string.**

**POTENSI RISIKO:**

**❗ Query masih berbentuk raw string**
**Capsule::raw($sql)**

**➡️ Artinya:**

- **Query tetap ditulis manual oleh developer**
- **Kalau developer bikin seperti ini:**

**$sql = "SELECT * FROM users WHERE username = '$username'";**
**➡️ 💥 tetap bisa kena SQL Injection**

**DAO tidak memaksa penggunaan prepared statement secara ketat, sehingga keamanan sangat bergantung pada implementasi di level DAO turunan.**

**Dynamic SQL Construction**
**Contoh:**
**$sql .= ' LIMIT ' . (int) $dbResultRange->getCount();**
**$sql .= ' OFFSET ' . $offset;**
**✔️ Aman karena: di-cast ke integer**

**TAPI:**
**'SELECT COUNT(*) FROM (' . $sql . ')'**
**SQL disusun ulang secara dinamis**

**Risiko:**
**kalau $sql berasal dari input tidak aman → bisa injection chain**

**Penggunaan unserialize()**
**$value = unserialize($value);**

**Ini bahaya:**
- bisa kena **Object Injection**
- **apalagi kalau data dari DB / user**

**Penggunaan unserialize() tanpa validasi dapat menyebabkan Object Injection apabila data berasal dari sumber tidak terpercaya.**

**Dynamic Table Name**
**Contoh:**
**"SELECT * FROM $tableName"**
**Kalau $tableName tidak dikontrol → risiko SQL Injection**

Pada file DAO.inc.php ditemukan bahwa sistem menggunakan parameter binding melalui $params pada saat eksekusi query, yang merupakan praktik aman untuk mencegah SQL Injection.

Namun demikian, query masih dibangun secara manual menggunakan raw SQL (Capsule::raw), sehingga terdapat ketergantungan pada developer untuk memastikan tidak terjadi concatenation dengan input user secara langsung.

Selain itu, ditemukan penggunaan fungsi unserialize() pada data dari database, yang berpotensi menyebabkan Object Injection apabila data tidak divalidasi dengan baik.

Secara umum, implementasi DAO cukup aman, namun masih memiliki risiko jika tidak digunakan dengan benar pada layer aplikasi di atasnya.

Hal ini menunjukkan bahwa keamanan pada layer database bersifat semi-safe, di mana framework menyediakan mekanisme aman, tetapi tidak sepenuhnya memaksa penggunaannya.

3. classes/submission/SubmissionFileManager.inc.php

Setelah dilakukan pencarian pada seluruh codebase, tidak ditemukan penggunaan langsung `$_FILES` maupun `move_uploaded_file()` pada folder `classes/submission/`. Hal ini menunjukkan bahwa mekanisme upload file tidak terletak pada layer tersebut, melainkan di "C:\Users\SYIFA\Documents\ojs-src\lib\pkp\classes\file\FileManager.inc.php".

**Entry point:**
**function uploadFile($fileName, $destFileName) {**
**...**
**if (move_uploaded_file($_FILES[$fileName]['tmp_name'], $destFileName))**

**titik upload sebenarnya**

**Validasi File:**
**Tidak ditemukan:**
**pathinfo(...)**
**in_array(...)**
**Artinya:**
**User bisa upload file apapun (termasuk .php)**

**ADA MIME CHECK — tapi lemah**
**PKPString::mime_content_type(...)**
**MASALAH:**

- **MIME bisa dipalsukan**
- **fallback ke:**

**$_FILES[$fileName]['type']**

**ini 100% bisa dimanipulasi**

**TIDAK ADA VALIDASI DI uploadFile()**
**function uploadFile(...) {**
**  move_uploaded_file(...)**
**}**
**langsung upload tanpa check apapun ❌**

**NAMA FILE TIDAK DISANITASI**
**$destFileName**
**langsung dipakai**
**Tidak ada:**
**basename()**
**preg_replace()**

**Risiko:**

- **Path Traversal (../../shell.php)**
- **Overwrite file penting**
- **RCE (Remote Code Execution)**

**EXTENSION CHECK ADA — TAPI TERPISAH**
**function parseFileExtension(...)**
**bahkan ada comment developer:**
**// FIXME Check for evil**

Validasi tidak dilakukan saat upload, tapi dipisah ke fungsi lain → rawan bypass

Pada file `FileManager.inc.php`, ditemukan bahwa proses upload file dilakukan menggunakan fungsi `move_uploaded_file()` tanpa validasi keamanan yang memadai.

Temuan

- Tidak terdapat validasi ekstensi file sebelum proses upload dilakukan.
- Tidak terdapat whitelist tipe file yang diizinkan.
- MIME type diperiksa menggunakan `mime_content_type()`, namun masih dapat dipalsukan dan

bahkan fallback ke `$_FILES['type']` yang tidak dapat dipercaya.
- Nama file tujuan (`$destFileName`) tidak disanitasi sebelum digunakan.
- Terdapat fungsi `parseFileExtension()`, namun tidak digunakan secara langsung pada proses upload dan bahkan mengandung komentar `FIXME`, yang menunjukkan bahwa validasi belum lengkap.

Risiko

- Upload file berbahaya seperti webshell (.php)
- Remote Code Execution (RCE)
- Path Traversal melalui manipulasi nama file
- Overwrite file penting pada server

Kesimpulan

Implementasi upload file pada `FileManager.inc.php` tidak memiliki validasi keamanan yang kuat. Validasi tidak dilakukan secara langsung pada proses upload, sehingga berpotensi membuka celah eksploitasi jika tidak dikontrol pada layer lain.

Rekomendasi

- Tambahkan validasi whitelist ekstensi file
- Gunakan validasi MIME type berbasis file content (bukan header)
- Sanitasi nama file sebelum disimpan
- Gunakan random filename (UUID)
- Tolak file dengan ekstensi berbahaya seperti `.php`, `.exe`, dll

4. plugins/generic/

**OUTPUT TANPA SANITASI → POTENSI XSS**
**echo $this->_getHTMLContents($request, $galley);**

**masalahnya ada di:**
**$contents = Services::get('file')->fs->read(...)**

**lalu langsung:**
**echo $contents;**

**File HTML dari user / submission langsung ditampilkan TANPA escaping.**
**Artinya:**
**kalau user upload HTML berisi:**
**<script>alert(1)</script>**
**akan dieksekusi → Stored XSS**

**MANIPULASI HTML TANPA SANITASI**
**$contents = preg_replace(...)**
**banyak replace:**

- **src**
- **href**
- **Url**

**tapi:**

- **tidak ada filtering script**
- **tidak ada sanitization**

**Risiko:**

**Attacker bisa inject HTML/JS berbahaya**

**TEMPLATE VARIABLE REPLACEMENT (RAW)**
**$contents = str_replace('{$' . $key . '}', $value, $contents);**

**$value berasal dari:**
**$journal->getLocalizedName()**
**$site->getLocalizedTitle()**
**tidak di-escape**

**Risiko:**
**XSS dari metadata / DB**

**FILE CONTENT DIRECT READ**
**$contents = Services::get('file')->fs->read(...)**
**file dibaca langsung**

**tidak ada:**

- **content validation**
- **sanitization**

**Plugin ini TRUST HTML CONTENT terlalu tinggi**
**Padahal:**
**HTML = bisa contain JS = XSS**

Pada plugin `htmlArticleGalley`, ditemukan beberapa potensi kerentanan keamanan yang berkaitan dengan pengolahan dan output HTML.

Temuan:
- Konten HTML dibaca langsung dari file menggunakan `Services::get('file')->fs->read()` tanpa validasi.
- Konten tersebut langsung ditampilkan ke user menggunakan `echo` tanpa proses sanitasi.
- Tidak terdapat mekanisme escaping atau filtering terhadap konten HTML yang ditampilkan.
- Penggunaan `preg_replace()` untuk memodifikasi HTML dilakukan tanpa validasi keamanan.
- Variabel template digantikan menggunakan `str_replace()` tanpa escaping, sehingga berpotensi menampilkan data tidak aman.

Risiko:

- Stored Cross-Site Scripting (XSS)
- Injection melalui konten HTML
- Eksekusi JavaScript berbahaya dari file yang diupload

Kesimpulan
Plugin ini mempercayai konten HTML secara langsung tanpa validasi atau sanitasi yang memadai, sehingga membuka peluang eksploitasi XSS jika attacker dapat mengontrol isi file HTML.

Rekomendasi
- Gunakan HTML sanitization library (misalnya HTMLPurifier)
- Hindari output langsung menggunakan `echo`
- Terapkan escaping pada semua data yang ditampilkan
- Batasi atau filter tag HTML berbahaya seperti `<script>`

5. lib/pkp/classes/template/PKPTemplateManager.inc.php

**HEADER HTML LANGSUNG DI-INJECT (NO SANITIZATION)**
**$this->addHeader('customHeaders', $customHeaders);**
**Sumber:**
**$customHeaders = $currentContext->getLocalizedData('customHeaders');**

**Masalah:**
**User / admin bisa inject HTML langsung ke <head>**

**contoh payload:**
**<script>alert(1)</script>**
**akan langsung masuk ke halaman**

**Impact:**

- **Stored XSS**
- **Full DOM compromise**
- **Cookie/session stealing**

**META TAG TANPA ESCAPING**

```
$this->addHeader(
  'searchDescription',
  '<meta name="description" content="' .
$currentContext->getLocalizedData('searchDescription') . '">'
);
```

**Masalah:**
**Data dimasukkan ke attribute HTML tanpa escaping**

**kalau input:**
**" onmouseover="alert(1)**

**Jadi:**

```
<meta content="" onmouseover="alert(1)">
```

**HTML HEADER BUILDER RAW STRING**
**function addHeader($name, $header, $args = [])**
**dia cuma:**

```
$this->_htmlHeaders[...] = [
  'header' => $header,
];
```

**tidak ada:**

- **sanitization**
- **validation**
- **escaping**

**Artinya:**
**Semua plugin / input bisa inject HTML arbitrary**

**JAVASCRIPT INLINE INJECTION**

```
$this->addJavaScript(
  'pkpLibData',
  $output,
  [
    'inline' => true,
  ]
);
```

**$output berisi:**

```
$output .= '$.pkp.app = ' . json_encode($app_data) . ';';
```

**Potensi masalah:**
**Kalau $app_data tidak bersih:**
**→ JS injection**

**Walaupun:**
**✔ pakai json_encode (ini relatif aman)**
**tapi tetap:**
**bergantung ke input sebelumnya**

**USER DATA MASUK TEMPLATE TANPA FILTER**

```
$this->assign([
  'displayPageHeaderTitle' => $currentContext->getLocalizedData('name'),
]);
```

**Masalah:**

**Data dari DB langsung dikirim ke template**

➡️ **kalau template tidak pakai escape:**

💥 **XSS**

**CUSTOM HEADER PALING BAHAYA**
**Ini yang paling critical:**

```
if (!empty($customHeaders)) {
  $this->addHeader('customHeaders', $customHeaders);
}
```

**ini berarti:**
**User bisa inject:**
**- <script>**
**- <iframe>**
**- <meta refresh>**
💥 **ini full XSS gateway**

**TemplateManager TRUST data terlalu tinggi**
**Masalahnya bukan di Smarty, tapi di:**
**data sebelum masuk template**

File `PKPTemplateManager.inc.php` berfungsi sebagai pengelola template berbasis Smarty dan bertanggung jawab atas rendering HTML di seluruh aplikasi.

Temuan:
- Fungsi `addHeader()` menerima input HTML mentah tanpa sanitasi atau validasi.
- Data `customHeaders` dari database dimasukkan langsung ke dalam `<head>` tanpa escaping.
- Meta tag seperti `searchDescription` dibangun menggunakan string concatenation tanpa escaping terhadap input user.
- Data dari database (misalnya nama jurnal, site title) langsung di-assign ke template tanpa sanitasi eksplisit.
- Script JavaScript inline di-generate menggunakan data dinamis, berpotensi menjadi injection point jika data tidak tervalidasi.

Risiko:
- Stored Cross-Site Scripting (XSS)
- HTML injection di bagian `<head>`
- JavaScript injection melalui inline script
- Full client-side compromise

Kesimpulan:
TemplateManager tidak melakukan sanitasi terhadap input sebelum dirender ke template, sehingga keamanan sangat bergantung pada validasi di layer lain. Hal ini membuka peluang eksploitasi XSS jika data yang disimpan di database dapat dimanipulasi oleh attacker.

Rekomendasi:
- Escape semua output menggunakan mekanisme Smarty (`|escape`)
- Validasi dan sanitasi input sebelum disimpan ke database

- Batasi penggunaan `customHeaders` atau filter tag berbahaya
- Gunakan Content Security Policy (CSP)